

Mini Project - 1 Report: Plant Leaf Disease Segmentation Dataset

1. Selection of Application

- **Problem Statement:** Automated detection of plant diseases is critical for scaling precision agriculture. Manual inspection is slow and error-prone. This project builds a dataset optimized for segmenting structural damage on leaves.
- **Purpose of Collection:** To gather diverse visual samples of plant leaves to isolate healthy tissue from lesions or infected areas.
- **Type of Images Required:** Standard digital RGB photographs of individual plant leaves over varying or uniform backgrounds.

2. Image Collection & Dataset Organization

- **Source:** Subsampled from the public **PlantVillage Dataset** repository.
- **Variation:** Includes diverse leaf structures, varying levels of disease spots, and differing lighting angles to ensure robust model generalizability.
- **Organization Index:** A comprehensive file named `dataset_summary.csv` has been systematically generated via automation. It maps every file (`1.jpg` to `100.jpg`) directly to its origin metadata, format (`JPEG`), and original image properties.

3. Preprocessing Pipeline & Engineering Rationale

The dataset was downsampled to exactly **100 images** and run through a programmatic pipeline using the following strict structural operations:

- **Grayscale Conversion:** Reduces the input image data from three color channels (RGB) to a single intensity channel. This eliminates color dependencies and simplifies computational complexity, isolating structural and tonal damage markers.
- **Noise Removal (Gaussian Blur):** Employs a (5×5) Gaussian kernel filter to smooth out high-frequency spatial noise and sensor grain. This prepares the image for clean edge boundaries without altering macro-geometric structures.
- **K-Means Clustering Segmentation ($K=5$):** Groups the grayscale pixel intensity array into exactly **5 structural intensity clusters**. This partitions background pixels, healthy leaf surfaces, mid-tones, and distinct lesion/disease regions cleanly into explicit segments.

- **80% JPEG Compression:** Balances high-fidelity data preservation with efficient storage optimization. It minimizes memory foot-print while maintaining sharp spatial pixel boundaries required for subsequent network training.

Example Image:

```
Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy in .\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages (2.2.4)
Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl (44.0 MB)
44.0/44.0 MB 25.0 MB/s 0:00:01

Installing collected packages: opencv-python
Successfully installed opencv-python-5.0.0.93
PS C:\Users\adhithya> cd REPOS
PS C:\Users\adhithya\REPOS> cdipmp
cdipmp : The term 'cdipmp' is not recognized as the name of a cmdlet, function, script file, or operable program.
Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ cdipmp
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (cdipmp:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\adhithya\REPOS> cd ipmp
PS C:\Users\adhithya\REPOS\ipmp> python process_100.py
Found 100 images in 'raw_images'.
Randomly selected exactly 100 images. Processing pipeline starting...

Success! Successfully processed 100 images.
Your final curated dataset is waiting in the folder: 'processed_dataset'
PS C:\Users\adhithya\REPOS\ipmp>
```

